Computer Vision

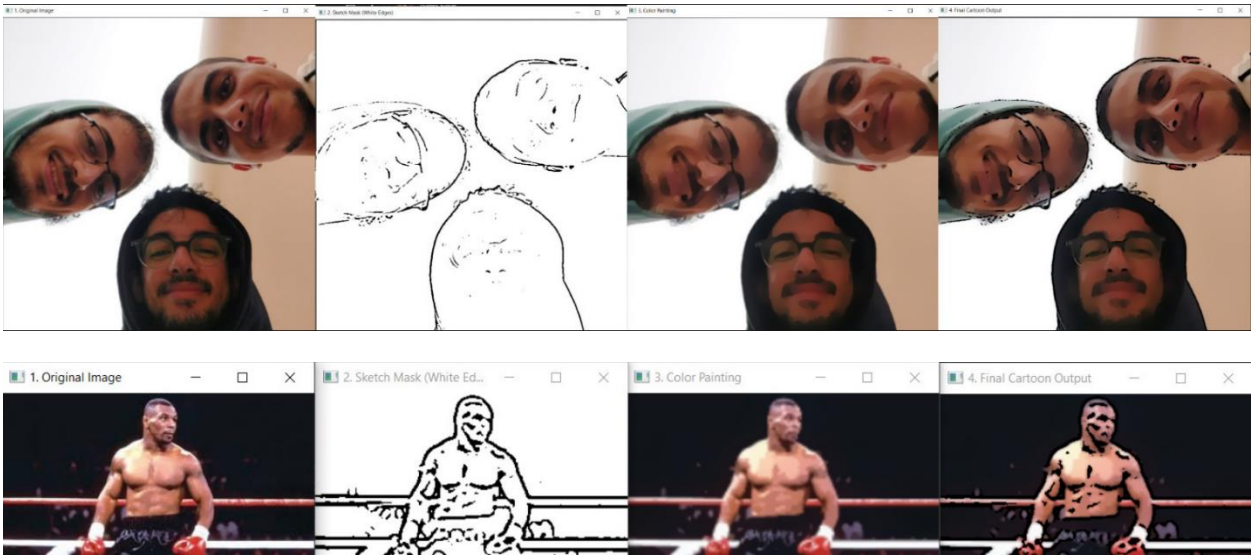
Assignment 1

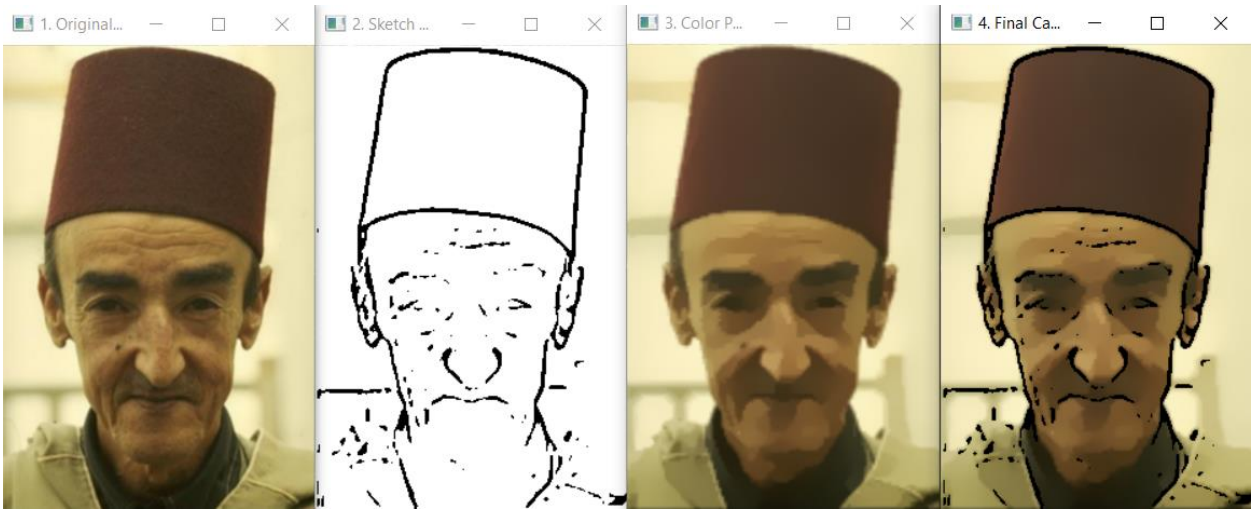
GitHub Repository Link

Team Members	IDs
Omar Hani Bishr	21010891
Muhammad Hassan Muhammad Kabary	21011115
Mohamed Mohamed Ali Hassan	21011211

Part 1 Image Cartoonifying

some test images





Code Explanation

The process is divided into three main functions:

1. `create_sketch_mask`

This function generates a binary edge mask from the original image.

- **Noise Reduction:** A **Median Filter** (`cv2.medianBlur`) is applied to the grayscale image to remove noise while preserving general edge-like structures.
- **Edge Detection:** A **Laplacian Filter** (`cv2.Laplacian`) is applied to the blurred image to detect edges. It uses a 16-bit signed data type (`cv2.CV_16S`) to accurately capture both positive (light-to-dark) and negative (dark-to-light) slopes.
- **Normalization:** The Laplacian output is scaled to a standard 8-bit (0-255) range using `cv2.normalize`.
- **Thresholding:** A binary threshold (`cv2.THRESH_BINARY_INV`) is applied. The `_INV` (inverse) flag is critical, as it creates the final mask with **black edges (value 0)** on a **white background (value 255)**.

2. `create_color_painting`

This function creates the smoothed, "painted" version of the image using an optimized bilateral filtering technique.

- **Optimization:** The original color image is first **downsampled** to a lower resolution (50%). This significantly speeds up the computationally expensive filtering step.
- **Smoothing:** A **Bilateral Filter** (`cv2.bilateralFilter`) is applied **repeatedly**. This flattens the colors in non-edge regions, creating the "painting" effect. Because it's a bilateral filter, it preserves the sharpness of the main edges.
- **Restoration:** The final smoothed, low-resolution image is **upsampled** back to the original image's dimensions.

3. `combine_with_sketch`

This function overlays the sketch on top of the painting.

- **Resizing:** The `sketch_mask` is resized to exactly match the `color_painting`'s dimensions to prevent any processing errors.
- **Overlay:** A **Bitwise AND** (`cv2.bitwise_and`) operation is performed. The `color_painting` is used as the input, and the `sketch_mask` is used as the `mask`. Because the mask has black (0) values for the edges and white (255) values everywhere else, this operation "stamps" the black lines onto the painting, creating the final cartoon image.

The `main()` function handles loading the images, defining all tunable parameters (kernel sizes, sigma values), and calling these functions in the correct order to produce and save the final output.

Part 2

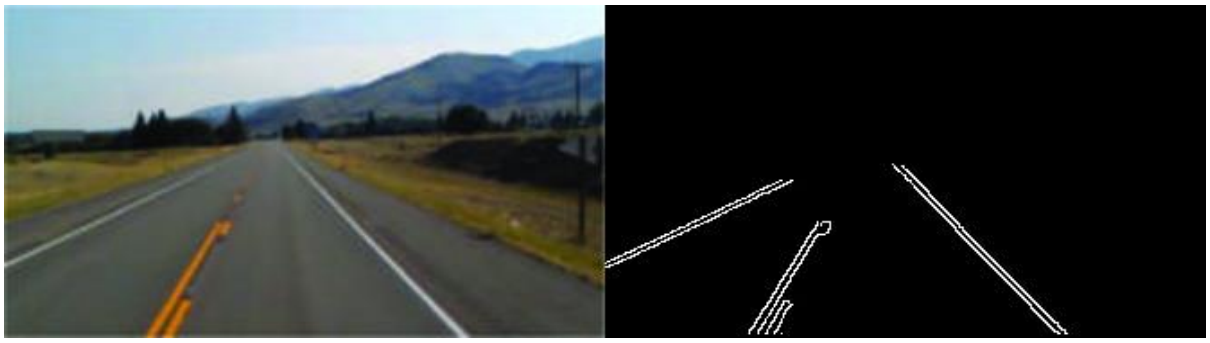
Lane Road Detector

Algorithm:

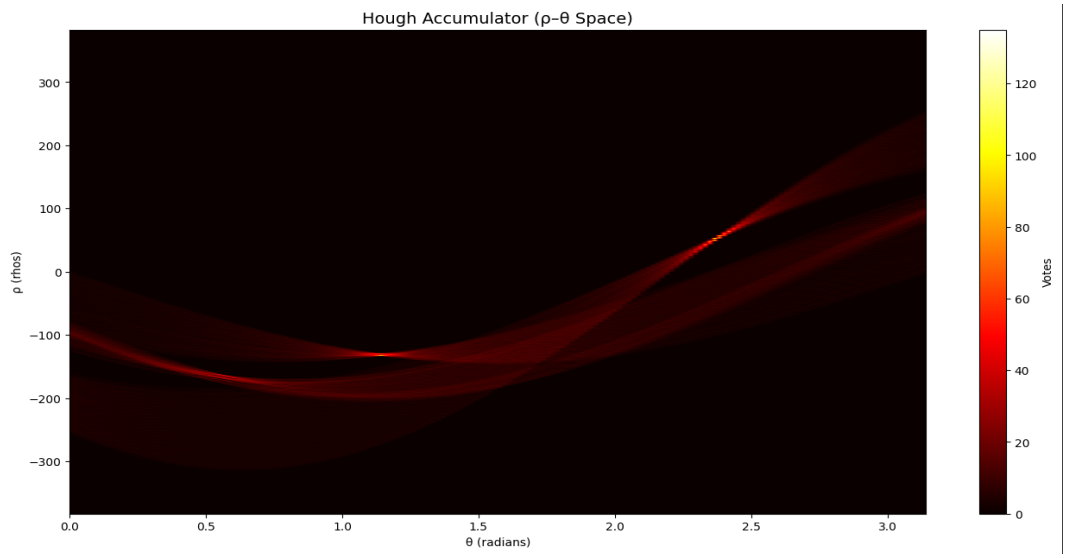
1. Apply 2d median filter to remove noise.
2. Apply canny detection and detect Roi.
3. Set step sizes for ρ (1 pixel) and θ (1°).
4. Compute the maximum possible ρ (image diagonal length).
5. Initialize the accumulator array for all (ρ, θ) combinations.
6. Precompute sin and cos values for efficiency.
7. Keep only strong edge pixels (above 10% of max edge value).
8. For each edge pixel, calculate ρ for every θ and update the accumulator.
9. Apply non maximum suppression to the accumulator (threshold = 30%, window = 7).
10. Convert detected (ρ, θ) pairs to lines and draw them on the image.
11. Find where the detected lines meet (intersection) and keep only the lower region.

Results:

Canny edge detection and Roi Detection:



Hough Space:



Hough Lines: